

Competitive Programming

Section 30: GCD, LCM, and Diophantine Equations

Competitive Programming Course

2026-07-10

Table of contents

1	Goals and problem signals	3
2	GCD: the invariant behind Euclid's algorithm	4
2.1	Trace: gcd(252, 105)	4
3	LCM: use the identity, but preserve the arithmetic order	5
3.1	Trace: a=84, b=126	5
4	Extended Euclid: construct Bezout coefficients	6
4.1	Trace: coefficients for 30 and 18	6
5	General modular inverse	8
5.1	Trace: inverse of 6 modulo 35	8
6	Linear Diophantine equations	9
6.1	Trace: solve $6x + 10y = 14$	9
7	Implementation notes	10
7.1	C++	10
7.2	Python	10
8	Failure modes and tests	11
9	Formative exercises	12
10	Exercises	13
11	Local workflow	14

1 Goals and problem signals

This section develops one chain of ideas:

```
Euclid computes gcd.  
Extended Euclid also constructs coefficients for the gcd.  
Those coefficients give modular inverses and solve  $ax + by = c$ .
```

The three local problems are deliberately close, but their outputs are different. Before coding, identify which question is being asked.

Statement signal	Mathematical object	First operation
“largest number dividing both”	$\gcd(a, b)$	Euclid
“smallest common multiple”	$\text{lcm}(a, b)$	divide by gcd before multiplying
“inverse modulo m ”	$a \cdot x + m \cdot y = 1$	extended Euclid, then normalize x
“find integers x, y ”	$a \cdot x + b \cdot y = c$	test $\gcd(a, b) \mid c$, then scale Bezout coefficients

The local inputs are positive integers. That keeps the implementation compact; for variants with negative inputs, first make the gcd nonnegative and take extra care with language-specific division and remainder rules.

2 GCD: the invariant behind Euclid's algorithm

For positive integers a and b , the greatest common divisor is the largest positive integer dividing both. Euclid's key identity is:

$$\gcd(a,b) = \gcd(b, a \bmod b)$$

Write $a = q*b + r$, where $r = a \bmod b$. A number divides both a and b exactly when it divides both b and $a - q*b = r$. Thus a Euclid step preserves the *set* of common divisors, not merely its greatest member.

Pseudocode:

```
while b != 0:
    (a, b) = (b, a mod b)
return a
```

2.1 Trace: $\gcd(252, 105)$

Current a	Current b	Remainder $a \bmod b$	Next pair
252	105	42	(105,42)
105	42	21	(42,21)
42	21	0	(21,0)

The algorithm returns 21. At every row, the common divisors are unchanged; when the second value is zero, every positive divisor of the first value is a common divisor, so that first value is the gcd.

Euclid runs in $O(\log \min(a,b))$ iterations. Consecutive Fibonacci numbers are the familiar near-worst case, so this bound remains excellent even for 10^{18} -scale values.

3 LCM: use the identity, but preserve the arithmetic order

For the positive inputs in problem A:

```
gcd(a,b) * lcm(a,b) = a*b
```

The identity follows prime by prime. If prime p occurs with exponents u and v in a and b , then gcd uses $\min(u,v)$ and lcm uses $\max(u,v)$. Their exponents add to $u+v$, exactly the exponent in $a*b$.

Compute:

```
g = gcd(a,b)
l = (a / g) * b
```

The parentheses are part of the algorithm. $a*b/g$ can overflow before the division even when the final lcm fits. Dividing first removes the common factor. It does **not** prove that the final product fits; use `__int128` when the constraints permit lcm above `long long`.

3.1 Trace: a=84, b=126

```
gcd(84,126) = 42
lcm(84,126) = (84 / 42) * 126 = 252
```

Check both directions: $252/84=3$ and $252/126=2$, so it is a common multiple; the gcd identity proves it is the least one.

4 Extended Euclid: construct Bezout coefficients

Ordinary Euclid returns only $g = \text{gcd}(a,b)$. Extended Euclid also returns integers x,y satisfying:

$$a*x + b*y = g$$

This is Bezout's identity. The recursive base case is clear:

$$\text{when } b = 0: \quad a*1 + 0*0 = a$$

For a non-base call, recursively obtain:

$$b*x_1 + (a \bmod b)*y_1 = g$$

Since $a \bmod b = a - \text{floor}(a/b)*b$, substitute it:

$$\begin{aligned} b*x_1 + (a - \text{floor}(a/b)*b)*y_1 \\ = a*y_1 + b*(x_1 - \text{floor}(a/b)*y_1) \end{aligned}$$

Therefore the parent call can return:

$$\begin{aligned} x &= y_1 \\ y &= x_1 - \text{floor}(a/b)*y_1 \end{aligned}$$

Pseudocode, including the exact return contract:

```
extended_gcd(a, b):  
    if b == 0:  
        return (a, 1, 0)  
    (g, x1, y1) = extended_gcd(b, a mod b)  
    return (g, y1, x1 - floor(a/b)*y1)
```

4.1 Trace: coefficients for 30 and 18

Back-substitution makes the construction visible:

$$\begin{aligned} 30 &= 1*18 + 12 \\ 18 &= 1*12 + 6 \\ 6 &= 30 - 18 \\ 6 &= 30 - 18 \end{aligned}$$

So one returned triple is:

```
g=6, x=1, y=-1  
30*1 + 18*(-1) = 6
```

There are many valid coefficient pairs. Extended Euclid is useful because it constructs one deterministically in logarithmic time.

5 General modular inverse

An inverse of a modulo m is a residue x with:

$$a*x \equiv 1 \pmod{m}$$

This congruence is equivalent to the existence of an integer y with:

$$a*x + m*y = 1$$

Bezout's identity says an integer linear combination of a and m can equal 1 exactly when $\gcd(a,m)=1$. Thus:

$$\text{inverse exists} \iff \gcd(a,m) = 1$$

When extended Euclid returns $a*x + m*y = 1$, the coefficient x is an inverse. Normalize it because judges normally require the smallest nonnegative representative:

```
x = x mod m
if x < 0: x += m          # needed in C++ after a negative remainder
```

The local inverse problem defines the answer for $m=1$ to be 0. Handle it before the ordinary inverse logic; modulo one has only the residue zero.

5.1 Trace: inverse of 6 modulo 35

Extended Euclid finds:

$$6*6 + 35*(-1) = 1$$

Hence 6 is the inverse. Verify the required congruence, not ordinary division:

$$6*6 = 36 \equiv 1 \pmod{35}$$

For 6 modulo 9, $\gcd(6,9)=3$. Every combination $6*x+9*y$ is divisible by three, so it can never equal one. Returning any number as an inverse is wrong.

6 Linear Diophantine equations

Consider:

$$a*x + b*y = c$$

Every left-hand side is a multiple of $g=\gcd(a,b)$, so a necessary condition is $g \mid c$. It is also sufficient. If extended Euclid gives:

$$a*x_0 + b*y_0 = g$$

and $c = g*scale$, multiply both sides by $scale$:

$$a*(x_0*scale) + b*(y_0*scale) = c$$

This yields one solution. The complete family is useful in harder variants with nonnegative or bounded variables:

$$\begin{aligned} x &= x_0 + (b/g)*t \\ y &= y_0 - (a/g)*t \end{aligned}$$

for any integer t . Substitution shows the additional terms cancel.

6.1 Trace: solve $6x + 10y = 14$

Extended Euclid may return:

$$6*2 + 10*(-1) = 2$$

Here $g=2$ divides 14, and $scale=7$:

$$\begin{aligned} x &= 2*7 = 14 \\ y &= -1*7 = -7 \end{aligned}$$

Verification is mandatory:

$$6*14 + 10*(-7) = 84 - 70 = 14$$

For $6x+10y=15$, the gcd is still two but does not divide fifteen, so no integer solution exists.

7 Implementation notes

7.1 C++

`std::gcd` is available through `<numeric>` (and therefore through the common contest include `bits/stdc++.h`). Store answers and Bezout coefficients in `long long` for the local 10^9 constraints. In larger variants, the update:

```
y = x1 - (a / b) * y1;
```

and the final scaling can require `__int128` even when `a` and `b` fit in `long long`.

7.2 Python

Use `math.gcd` when only the gcd is needed. Python integers do not overflow, but recursive extended Euclid still needs its base case and exact quotient formula. Python normalizes `x % m` into `[0, m-1]` for positive `m`; C++ does not, which is why the explicit correction is shown above.

8 Failure modes and tests

Mistake	Minimal counterexample	Correct reaction
compute $a*b/g$ first	large values with a small gcd	divide by gcd first
assume every a has an inverse	$a=6, m=9$	print -1 because gcd is 3
use Fermat under a composite modulus	inverse modulo 35	use extended Euclid
scale before divisibility test	$6x+10y=15$	report impossible
reject negative coefficients	$6x+10y=14$ gives $y=-7$	integers may be negative
forget modulus one	$a=25, m=1$	output 0 in the local contract

For every Diophantine output, substitute the printed values back into the original equation. This catches swapped coefficients, missing scale factors, and sign errors immediately.

9 Formative exercises

1. Compute $\gcd(391, 299)$ by Euclid. What invariant remains true after every remainder step?

Hint: write $391 = 1 \cdot 299 + 92$ and continue.

Answer: the gcd is 23; every step preserves the set of common divisors.

2. Does 14 have an inverse modulo 35? Explain without trying candidate residues.

Hint: an inverse would make an integer combination equal to one.

Answer: no. $\gcd(14, 35) = 7$, so every combination is divisible by seven.

3. Find one pair solving $15x + 21y = 12$.

Hint: first find a Bezout representation of $\gcd(15, 21)$.

Answer: $15 \cdot 3 + 21 \cdot (-2) = 3$; scale by four to get $(x, y) = (12, -8)$.

4. Describe all solutions to $6x + 10y = 14$ from the worked trace.

Hint: use $g=2$ and add a multiple of $(b/g, -a/g)$.

Answer: $x=14+5t, y=-7-3t$ for every integer t .

10 Exercises

Complete the four formative exercises above before implementing local Problems A–C; their hints and answers are deliberately included for self-checking.

11 Local workflow

Run the section checker after implementing each problem:

```
CP_TARGET=solution python3 sections/30_gcd_lcm_diophantine/check.py
```

Before submission, check these exact questions:

1. For lcm, did I divide before multiplying?
2. For an inverse, did I test `gcd(a,m) == 1` and normalize the coefficient?
3. For $ax+by=c$, did I test `c % gcd(a,b)` before scaling?
4. Did I verify $a*x+b*y$ on a small trace, including a negative coefficient?
5. Do my integer types cover intermediate products under the stated bounds?